

EcoreGenTL: A Hybrid Approach Fusing MDE Template-Based and LLM-Driven Code Generation

Abstract—Template-based code generation (TBCG) in model-driven engineering (MDE) produces structurally correct, compilable code but leaves incomplete logic that developers must fill manually. Large language models (LLMs) offer completeness yet often hallucinate, especially with multi-class or project-scale code. Our key insight is that the two paradigms are complementary. We propose EcoreGenTL, a hybrid approach that fuses TBCG with LLMs: MDE templates provide a sound skeleton, and LLMs act as intelligent fillers for missing method bodies. Given an Ecore model and a natural language requirement, EcoreGenTL first decomposes the requirement into fine-grained operation specifications. It then uses EMF’s template-based generator to produce Java skeletons with these specifications embedded as Javadoc strings. LLMs fill the method bodies guided by the docstrings, supported by code compression, context-retrieval tools, and AST-based merging that replaces only target bodies—preventing corruption of the skeleton. An iterative compilation-fixing loop further ensures correctness. Evaluated on JavaBench and a private benchmark with five LLMs, EcoreGenTL achieves 91.1% pass rate on JavaBench (vs. 74.2% for the best baseline) and 98.3% on our private benchmark TLBench, consistently outperforming agentic and vanilla LLM baselines. Ablation confirms that every component of EcoreGenTL contributes meaningfully to its performance. EcoreGenTL demonstrates how legacy MDE infrastructures can be augmented with LLMs without rewriting, and offers a new paradigm for LLM-based software engineering where models anchor generation to reduce hallucination.

Index Terms—code generation, model-driven engineering, large language models, code template

I. INTRODUCTION

Model-driven engineering (MDE) [1] has been a pivotal approach for automating software development [2] over the past two decades. In MDE, developers first create high-level software models describing a system’s structure and behavior. The implementation is then automatically generated through model transformation and code generation [3], [4]. MDE has been successfully applied across many domains [5]–[8] and forms the technical basis for low-code development [9].

Code generation in MDE is typically achieved using *code templates*—programs that algorithmically transform model elements into code by populating a predefined program structure. Figure 1 illustrates *template-based code generation* (TBCG) from a UML class named Employee to a Java class using the Xtend template `generateClass`. This template correctly generates structural features (fields, getters, setters) but leaves method bodies like `computeLongServiceBonus()` empty.

TBCG thus suffers from the *incompleteness* issue: it can only produce code that fits its predefined patterns, leaving gaps for developers to fill manually. In parallel, large language

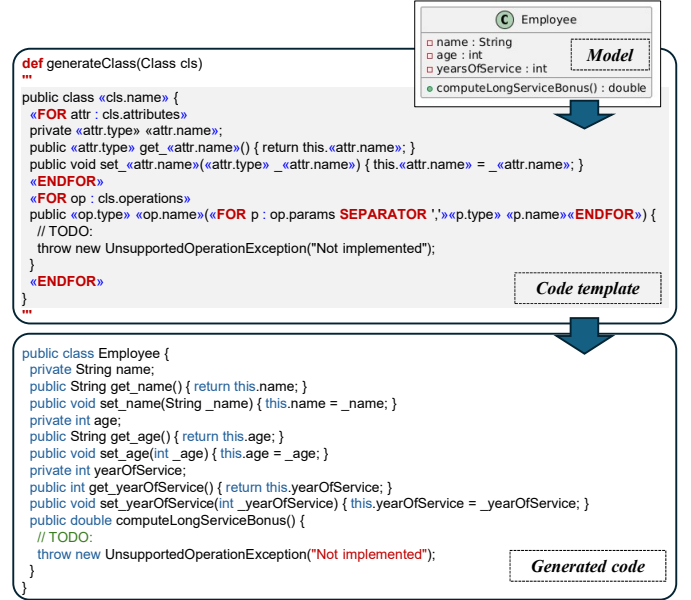


Fig. 1: Code generation in MDE

models (LLMs) have recently revolutionized software engineering (SE) [10] by demonstrating remarkable capabilities in generating code from natural language (NL) requirements [11], [12]. Unlike TBCG, LLMs are effectively complete: they always produce a response for any given request. Although LLMs perform well on standalone statements and functions, they struggle with multi-class or project-scale code [13], which requires reasoning across many modules within a large context. This limitation exacerbates LLMs’ *hallucination* tendency [14], frequently resulting in non-functional or incorrect code.

Based on observation, a compelling question emerges: *Can we strike a graceful equilibrium between the correctness of templates and the completeness of LLM?* To answer this question, this paper introduces EcoreGenTL, a novel method that combines MDE template-based and LLM-driven code generation to leverage the strengths of both paradigms. EcoreGenTL takes as input an Ecore model, a simplified class diagram from the Eclipse Modeling Framework (EMF, [15]), along with a NL requirement. It first collects all operations requiring implementation and decomposes the system requirement into fine-grained, operation-level specifications. Then, it uses EMF’s template-based code generator to produce Java code, serializing these specifications as Javadoc strings. LLMs subsequently fill the method bodies based on these docstrings

and the contextual information from related Java classes, while adhering to EMF’s coding style. When compilation errors arise, EcoreGenTL iteratively asks LLMs to fix the faulty code until it compiles successfully or the retry limit is reached.

In essence, EcoreGenTL uses the structurally sound code skeleton produced by TBCG as a foundation, and the LLM as an intelligent filler that completes missing logic, a symbiotic design that preserves the best of both worlds. To the best of our knowledge, EcoreGenTL is the first **systematic, end-to-end pipeline** that fuses MDE template-based and LLM-driven code generation at the code level. This hybrid approach opens new directions for both communities: for MDE, it shows how legacy TBCG can be augmented with LLMs to address the incompleteness problem without rewriting existing infrastructures; for LLM-based SE, it shows how the software models can harness LLMs to mitigate hallucinations, anchoring LLM generation in the MDE-produced code skeleton.

We evaluated EcoreGenTL against agentic baselines (OpenCode, SWE-Agent-Mini) and two vanilla LLM baselines on JavaBench [13] and a private benchmark, using five LLMs (including DeepSeek-V4-Flash, Qwen3.6-35B-A3B, MiniMax-M3, Gemini-3.1-Flash-Lite, and GPT-5.4-Mini). EcoreGenTL consistently achieved higher compilation and test passing rates than all baselines across both benchmarks and all LLMs we tested. We also performed an ablation study comparing EcoreGenTL to its ablated variants. The ablation results confirmed that each component of EcoreGenTL contributes meaningfully to its overall performance for complex tasks, with the most substantial drop observed when the requirement decomposition phase or code context retrieval tool is disabled.

This paper’s main contributions are summarized as follows:

- We propose **EcoreGenTL**, an approach that systematically fuses MDE template-based generation with LLM-driven completion, achieving a *correctness-completeness* trade-off that neither paradigm can reach alone.
- We implement a **prototype tool** atop EMF/Ecore, showing how existing MDE tools can be retrofitted with LLMs.
- We report an **empirical evaluation** to demonstrate the promise of this hybrid code generation paradigm.

Paper Structure. Section II revisits existing studies. Section III details the pipeline of EcoreGenTL. Section IV briefly introduces the prototype tool. Section V outlines the experimental process and analyzes the results. The final section provides conclusions and suggests directions for future research.

II. BACKGROUND AND RELATED WORK

A. Model-Driven Engineering

Model-driven engineering (MDE) is a model-centric software development paradigm [1]. In MDE, models like class diagrams and state charts are defined first and then used to analyze and verify software. The most promising feature of MDE is the automatic generation of code from models, which streamlines the development process.

MDE code generation is typically template-based [3]. A code template, e.g., `generateClass` in Figure 1, algorithmically traverses the input model and replaces placeholders

with extracted values. While templates enable correct and efficient code generation, they are insufficient when the required code deviates from predefined patterns. EMF [15]—the most popular MDE framework—generates Java code from an Ecore model, a simplified class diagram supporting classes, attributes, references, inheritance, and operations. By default, EMF will generate a placeholder body for each operation.

B. Related Work

The LLM-based coding paradigm has attracted substantial research attention in recent years. Generative neural networks, such as Deepseek [16], have demonstrated remarkable capabilities in many code-related tasks, including code generation [11], [12], [14], test generation [17], and bug fixing and repair [18], [19]. Existing LLM-based code generation methods typically take NL requirements or specifications as input [20] and iteratively translate these specifications into code.

The integration of MDE and AI has also gained significant attention [21], [22]. LLMs have been applied to various modeling tasks [23], such as model completion [24]–[27], model generation [28]–[33], model correctness checking [34], [35], model plagiarism checking [36], model query generation [37], and model-based reverse engineering [38]. This paper focuses on MDE code generation rather than modeling.

Several recent efforts have explored the use of LLMs within MDE code generation. The first paradigm replaces the traditional template-based generator with an LLM. Sadik et al. [39] used LLMs to generate source code directly from UML and OCL models converted into text. Similarly, Siavash [40] proposed using LLMs together with Retrieval-Augmented Generation (RAG) to generate quantum code from models.

In contrast, the second paradigm retains the MDE template-based generator while using LLMs to create the input models from natural language. Malamas et al. [41] integrated a conversational LLM agent into a low-code platform to generate DSL models from NL prompts, which are then validated and transformed into deployable applications. Yang et al. [42] used an LLM to generate DSL models from NL requirements, feeding them into a template-based generator for HarmonyOS code. Al-Azzoni et al. [43] asked an LLM to generate model-based test cases from a role-based access control model and an NL requirement, then converted them into source code via an existing MDE code generator.

Different from previous efforts, which either replace templates with LLMs or use LLMs only for model creation, EcoreGenTL integrates both at the code level—templates generate the skeleton and LLMs fill in the missing parts.

We note that MDE for LLM has also been explored, though it is orthogonal to our focus. Morales et al. [44] presented an MDE framework for managing prompts for generative AI. Chen et al. [45] designed a model-driven framework that incorporates domain-specific best practices into hierarchical state machines to enhance LLM execution.

III. APPROACH

This section presents EcoreGenTL, as shown in Figure 2. EcoreGenTL is an automated pipeline that combines template-

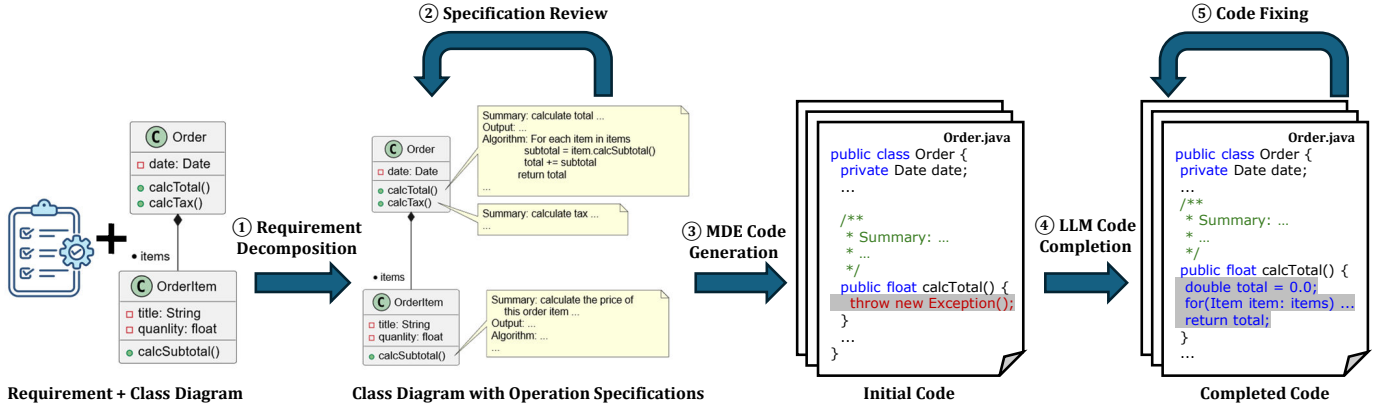


Fig. 2: Approach overview

based and LLM-driven code generation, differing from agent-based approaches [46]. The input of this pipeline is an Ecore model and a natural language requirement for a system under development, and its output is Java source code. EcoreGenTL consists of five major steps, as described below.

A. Requirement Decomposition

EcoreGenTL takes an Ecore model and an NL requirement as input. The requirement provides a holistic description of the system under development, while the Ecore model formally defines the classes and their relationships in the system. Each class may own some operations to be realized.

Because we do not assume that the NL requirement specifies every operation in detail, the first step of EcoreGenTL is to refine and decompose the input requirement into an operation specification for each operation. Algorithm 1 describes how EcoreGenTL generates operation specifications with LLMs. EcoreGenTL first converts the Ecore model into a textual representation¹ and collects all operations defined in the model. It then iteratively prompts an LLM to generate specifications for all operations by analyzing and decomposing the requirement. Because the LLM might neglect some operations, the process repeats until every operation is processed.

Algorithm 1: Generate operation specifications

Input: M —an Ecore model; R —an NL requirement
Output: $op2spec$ —a map from operations to specifications

```

1  $puml \leftarrow to\_text\_representation(M)$ ;
2  $ops \leftarrow$  all operations defined in  $M$ ;
3  $op2spec \leftarrow \emptyset$ ;
4 while  $ops \neq \emptyset$  do
5    $prompt \leftarrow gen\_spec\_prompt(R, puml, ops)$ ;
6    $op2spec \leftarrow op2spec \cup call\_llm(prompt)$ ;
7    $ops \leftarrow ops - key(op2spec)$ ;
8 end
9 return  $op2spec$ ;

```

The function $gen_spec_prompt(\cdot)$ constructs the prompt text by combining the NL requirement R , the model representation

¹In our approach, an Ecore model is converted into PlantUML, by the function $to_text_representation(\cdot)$, before being sent to LLMs.

$puml$, and the remaining operations ops . The prompt requires an LLM to derive operation specifications by considering the principles of modularization, determining functional boundaries, and ensuring high cohesion and low coupling. To better guide LLMs, we provide a format for operation specifications in the prompt text, which outlines the key aspects needed for implementation, as specified below.

- **Summary**—the summary of the functionality.
- **Algorithm**—the algorithm or pseudo-code.
- **Input**—the definition of input parameters.
- **Output**—the definition of the output value.
- **Related Features**—a list of structural or behavior features within the same class that shall be accessed.
- **Related Classes**—a list of classes and data types defined in the model that shall be referenced.
- **Pre-condition**—the condition that must be satisfied before the operation is executed.
- **Post-condition**—the condition that must be satisfied after the operation is executed.

In this specification, the algorithm, input, and output sections directly support code generation, while the related features and classes sections provide essential context; the pre- and post-conditions improve robustness for edge cases.

For every operation, we store the obtained specification in the Ecore model as a documentation element that is associated with the corresponding operation.

B. Specification Review

We observe that large language models may exhibit hallucinations when generating operation specifications—for example, omitting, misinterpreting, or even reversing requirements.

Figure 3 illustrates an error in requirement decomposition. The requirement states that an IPO application must only be accepted if the number of shares is strictly greater than zero and all eligibility rules pass. However, the specification produced by the LLM neglects these input-validation checks, resulting in incorrect code that blindly creates and persists an Application object even when shares is zero or negative, causing the system to fail catastrophically on invalid input.

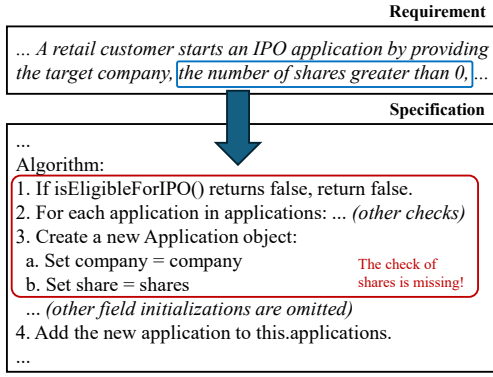


Fig. 3: An example of requirement decomposition error

To mitigate these issues, EcoreGenTL incorporates a second step: specification review. As shown in Algorithm 2, EcoreGenTL first converts the Ecore model into a textual representation and retrieves all classes defined in the model. For each class, it collects the existing specifications of the class’s operations. It then prompts an LLM to review these specifications, correcting any omissions, misinterpretations, or reversed requirements. This process repeats for every class, yielding a final set of reviewed operation specifications.

Algorithm 2: Review operation specifications

Input: M —an Ecore model; R —an NL requirement
Output: $op2updSpec$ —a map from operations to updated specifications

```

1  $puml \leftarrow to\_text\_representation(M)$ ;
2  $classes \leftarrow$  all classes defined in  $M$ ;
3  $op2updSpec \leftarrow \emptyset$ ;
4 foreach  $c \in classes$  do
5    $op2spec_c \leftarrow$  the map from the operations defined in
   class  $c$  to their specifications stored in  $M$ ;
6    $prompt \leftarrow rev\_spec\_prompt(R, puml, op2spec_c)$ ;
7    $op2updSpec \leftarrow op2updSpec \cup call\_llm(prompt)$ ;
8 end
9 return  $op2updSpec$ ;
```

The function $rev_spec_prompt(\cdot)$ constructs the prompt text by combining the requirement R , the model representation $puml$, and the operation-specification map $op2spec_c$ for class c , instructing the LLM to correct each specification against R .

After collecting all revised specifications, EcoreGenTL will update the Ecore model to reflect the changes.

C. MDE Code Generation

The third step is code generation using EMF’s template-based code generator. EcoreGenTL invokes the EMF code generator to translate an Ecore model with operation specifications into initial Java code. The generator automatically produces Java classes, fields, and auxiliary code such as getters, setters, and a creation factory. However, it treats all operations as unimplementable by default—generating only method signatures with empty bodies, while serializing the operation specification (i.e., documentation elements) as a docstring, as shown in Figure 4.

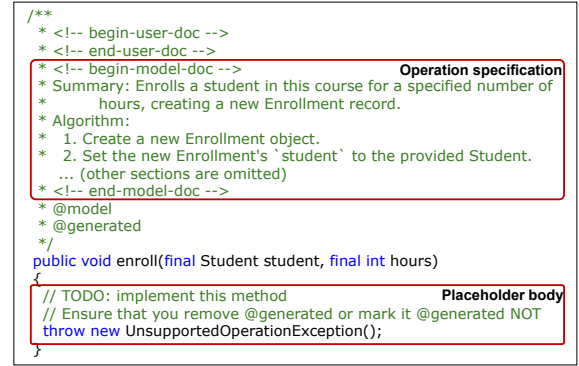


Fig. 4: An example of default operation implementation

Typically, EMF generates both a Java interface and an implementation class for each Ecore class. EcoreGenTL simplifies this by enabling the *suppressInterfaces* switch, instructing the generator to produce only a Java class.

D. LLM-Driven Code Completion

After EMF generates the initial Java code (with empty method bodies but filled-in docstrings), EcoreGenTL invokes an LLM to implement the unimplemented methods. The challenge here is twofold: the LLM must respect EMF’s peculiar coding conventions (e.g., lazy initialization of collections via getters) and avoid corrupting the structurally sound code skeleton produced by the template-based generator. Algorithm 3 presents the fourth step of EcoreGenTL.

Algorithm 3: Code completion

Input: M —an Ecore model

```

1  $classes \leftarrow$  all classes defined in  $M$ ;
2 foreach  $c \in classes$  do
3    $ops_c \leftarrow$  all operations defined in  $c$ ;
4    $file_c \leftarrow$  the Java source file path corresponding to  $c$ ;
5    $code_c \leftarrow read\_file(file_c)$ ;
6    $c\_code_c \leftarrow compress(code_c, ops_c)$ ;
7    $code\_tools \leftarrow$  new code tools;
8    $prompt \leftarrow completion\_prompt(c\_code_c)$ ;
9    $c\_code'_c \leftarrow call\_llm(prompt, code\_tools)$ ;
10   $code'_c \leftarrow ast\_merge(code_c, c\_code'_c, ops_c)$ ;
11   $save\_file(file_c, code'_c)$ ;
12 end
```

For each class c in the Ecore model M , the algorithm retrieves its operations (ops_c) and locates the corresponding Java source file generated by EMF. It reads the source code $code_c$ from the file and applies a function $compression(\cdot)$ to reduce token usage while sharpening the LLM’s focus on relevant code. A completion prompt is built from the compressed code c_code_c and sent to the LLM, which returns a completed version $c_code'_c$. The algorithm then merges $c_code'_c$ into the original $code_c$ using an AST-based merger, replacing only the method bodies of the target operations. Finally, the merged code is saved.

Compression: EMF-generated files contain many auxiliary elements (e.g., docstrings of fields, method bodies of unrelated operations) that are not needed for completing a specific set of target methods. Removing them reduces the prompt length and, more importantly, prevents the LLM from being distracted by irrelevant patterns. Hence *compress(·)* removes docstrings and comments from all fields, as well as docstrings and method bodies from methods that are not among the target operations *ops_c*.

Prompt construction and coding knowledge: The function *completion_prompt(·)* builds a prompt from the compressed code *c_code_c*. It explicitly tells the LLM which methods must be completed and where the corresponding operation specifications (the pre-generated docstrings) reside. However, a central challenge is that EMF’s template-based generator enforces a specific coding style that is unfamiliar to general-purpose LLMs. For example, EMF uses lazy initialization for collection fields: a field is not instantiated until its getter is called. Accessing such a field directly would lead to a null-pointer exception. To overcome this, *completion_prompt(·)* injects a set of EMF-specific coding rules into the prompt, as illustrated in Figure 5. The knowledge mainly concerns coding conventions and API usage patterns specific to EMF-generated code. We do not claim that the MDE knowledge we provided is comprehensive or generic; different MDE tools impose different coding conventions. Extracting such knowledge automatically from existing MDE repositories is promising future work.

1. Use getter and setter methods to access field values; do not directly read or write raw fields derived from the class diagram.
2. For a boolean attribute, the getter follows the pattern `is<AttributeName>()`, e.g., `isIsFull()` for attribute `isFull`.
3. Always use the creation factory to instantiate objects.
4. To create an `EList` instance, import and use `org.eclipse.emf.common.util.BasicEList`.

Fig. 5: MDE coding knowledge for LLM-driven code completion (excerpt)

Context retrieval: Beyond static information, the LLM often needs to understand the functionality of other classes (e.g., field types, method signatures). Therefore, we provide two code tools when invoking *call_llm(·)*: one lists all Java source files in the project; the other generates a compact summary of a given file. The LLM is instructed to retrieve summaries of related files before completing a method. A summary includes the declarations of public fields, the signatures of public methods, and the qualified name of the superclass. If the file corresponds to an Ecore class, the summary also contains the factory method name from the creation factory.

AST-based merging: A naïve approach would replace the entire original code *code_c* with the LLM-generated version *c_code_c*. This is unsafe because the LLM may inadvertently alter existing fields or methods. Instead, EcoreGenTL uses *ast_merge(·)* to preserve the original code structure while

incorporating only the intended changes. Concretely, the function *ast_merge(·)*:

- 1) Copies new import declarations from *c_code_c* to *code_c*;
- 2) Replaces the method bodies of target operations (*ops_c*) in *code_c* with the corresponding bodies from *c_code_c*;
- 3) Copies any newly generated fields or methods from *c_code_c* to *code_c*.

This way, all other parts of the file (e.g., unchanged getters and setters) remain exactly as EMF produced them, guaranteeing that the LLM’s contributions are confined to the missing logic.

E. Code Fixing

Because LLMs may introduce errors during code completion, the final step of EcoreGenTL is code fixing. Algorithm 4 outlines this error-driven, iterative fixing process. Unlike the proactive specification-based completion of Algorithm 3, this step reacts to compilation errors and repeatedly fixes only the faulty code until the project compiles successfully.

Algorithm 4: Code fixing

```

1 files ← all the source files of the project;
2 errors ← compile(files);
3 while errors ≠ ∅ do
4   err_files ← all files containing compilation errors;
5   foreach fe ∈ err_files do
6     codee ← read_file(fe);
7     err_infoe ← error information related to file fe;
8     err_methe ← error methods in fe;
9     c_codee ← compress(codee, err_methe);
10    prompt ← code_fix_prompt(c_codee, err_infoe);
11    code_tools ← new code tools;
12    c_codee ← call_llm(prompt, code_tools);
13    codee ← ast_merge(codee, c_codee, err_methe);
14    save_file(fe, codee);
15  end
16  errors ← compile(files);
17 end

```

First, the algorithm compiles all source files of the project and collects the resulting compilation errors. While errors remain, it identifies the set of files that contain errors. For each such file *f_e*, it reads the source code *code_e*, extracts the error messages *err_info_e*, and determines which methods are involved in the errors (*err_meth_e*). It then applies the same compression function *compress(·)* as in code completion, but this time focusing on the error methods. The compressed code *c_code_e* is passed to *code_fix_prompt(·)*, which builds a prompt that includes both the code and the relevant error information, instructing the LLM to correct the faults. The LLM, assisted by the same code tools as before, returns a fixed compressed version *c_code_e*. An AST-based merge (*ast_merge*) integrates the changes back into the original file, replacing only the bodies of the error-related methods. The updated file is saved, and after processing all erroneous files, the project is recompiled. The loop continues until no compilation errors remain.

IV. TOOL IMPLEMENTATION

We implemented a prototype tool based on the Eclipse Modeling Framework, the Xtext Modeling Workflow Engine (MWE), and Spring AI framework. It takes a `.mwe2` workflow configuration file as input, which specifies code generation parameters such as the Ecore model path, root folder, and project name. The tool then automatically generates source code by executing the full EcoreGenTL pipeline, with a fixed retry limit of 3. In the supplemental file, we provide the binary of EcoreGenTL and a simple showcase project.

V. EVALUATION

This section evaluates the effectiveness of EcoreGenTL, measured by the compilation success rate and test case pass rate, from the perspective of SE researchers, through controlled experiments against external baselines (vanilla LLM-based and agentic approaches) and internal ablation variants.

The research questions are described as follows.

- RQ1** How does EcoreGenTL compare to vanilla LLM-based and agent-based baselines in terms of compilation success rate and test case pass rate?
- RQ2** What is the individual contribution of each core component to the overall performance of EcoreGenTL?
- RQ3** What are the typical failure cases that EcoreGenTL cannot resolve, and what are their root causes?

We employ the following metrics to quantitatively assess the performance of a certain code generation approach.

- *Comp@k* [13] is the success rate of generating at least one compilable program out of k samples, calculated as Equation (1), where n is the total number of generated samples and c is the number of compilable ones.

$$\mathbb{E}_{Problems} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \quad (1)$$

- *Pass@k* is the probability that at least one of the k generated samples passes all test cases. Its formula is identical to Equation (1), with c now denoting the number of test-passing samples.
- *Pass%* is the expected value over problems of the total passed test cases divided by the total test cases for all n samples of that problem, as defined in Equation (2). It complements *Pass@k* by measuring the average fraction of test cases passed across samples, capturing partial correctness that the all-or-nothing *Pass@k* ignores.

$$\mathbb{E}_{Problems} \left[\frac{\sum_{i=1}^n p_i}{nT} \times 100\% \right] \quad (2)$$

where p_i is the number of test cases passed by the i -th sample, and T is the number of test cases for a problem. Note that a sample that is not compilable is considered to have failed all test cases.

- *C-Pass%* is the test case pass rate for all compilable samples, which is calculated as Equation (3), where T_j

denotes the number of compilable samples for a problem and $p_{n_1} \dots p_{n_j}$ are compilable samples.

$$\mathbb{E}_{Problems} \left[\frac{\sum_{i=n_1}^{n_j} p_i}{jT} \times 100\% \right] \quad (3)$$

We evaluate our approach on two benchmarks. Table I illustrates the general information on these benchmarks.

- The first is JavaBench [13], a project-level Java benchmark for LLM-driven code generation. It comprises four problems, each requiring the generation of multiple methods within several Java classes. In this benchmark, each Java class is passed to an LLM, which completes the empty methods based on their Javadoc comments. However, our evaluation paradigm requires a class diagram (e.g., an Ecore model) and a holistic requirement per problem, rather than a set of Java classes. To bridge this gap, we adapted each JavaBench problem as follows:
 - 1) We derived a class diagram from the provided Java classes, storing it as both an Ecore model and a PlantUML file;
 - 2) We created a holistic system requirement by aggregating all Javadoc strings and refining them with an LLM for readability, deliberately omitting explicit mappings from requirements to operations;
 - 3) We used (and adapted, if necessary) the test cases provided in JavaBench.
- The second is a private benchmark (called TLBench in this paper) constructed by the authors. It comprises 10 problems, constructed through the following procedure:
 - 1) Domain descriptions for the ten problems were initially collected from existing benchmarks [47], [48] for object-oriented domain modeling;
 - 2) We enhanced these domain descriptions by appending functional requirements, yielding 10 system requirements (i.e., combinations of domain descriptions and functional requirements);
 - 3) We created a class diagram (as both an Ecore model and a PlantUML file) for each system requirement;
 - 4) We developed test cases for each problem to validate implementation of the functional requirements;
 - 5) For every problem, at least two authors reviewed and discussed it to resolve any disputes.

A. RQ1: Overall Effectiveness

To answer RQ1, we compare EcoreGenTL with two vanilla LLM-based baselines and two agentic baselines. The details of these baselines are summarized as follows.

- **VL** is a vanilla LLM-based baseline with the simplest prompting strategy. It takes the NL requirement and the class diagram (in PlantUML format) as input and asks an LLM to generate Java code according to the class diagram to realize the requirement.
- **VL-Fix** is a vanilla LLM-based baseline that enhances **VL** with a code-fixing loop. It first generates Java code as **VL**, then iteratively fixes it if compilation errors occur.

TABLE I: General information of the benchmarks

Benchmark	#Problems	Requirement Complexity		Code Complexity			Covered Domains (Number of Cases)
		#Sentences	#Words	#Classes	#Methods	#Test Cases	
JavaBench	4	32–86	533–1128	13–29	32–53	45–145	Games (4)
TLBench	10	25–48	264–664	5–11	7–13	25–33	Transportation (3), Education (4), Business (2), Finance (1)

- **OpenCode**² is an open-source AI coding agent. In our tests, we provide an NL requirement along with a class diagram in PlantUML format to generate Java code.
- **SWE-Agent-Mini** [49] is an open-source agentic coding framework. Similarly, in our tests, we provide a natural language requirement and a class diagram in PlantUML format for Java code generation.

We evaluate EcoreGenTL on five state-of-the-art large language models, covering both open-source and closed-source models, as listed in Table II. We turn off reasoning mode and set the temperature to 0.7 for all LLMs.

TABLE II: Summary of selected large language models

Name	Category	Size	Release Time
MiniMax-M3	open-source	428B	June, 2026
DeepSeek-V4-Flash	open-source	284B	April, 2026
Qwen3.6-35B-A3B	open-source	35B	April, 2026
Gemini-3.1-Flash-Lite	closed-source	N/A	March, 2026
GPT-5.4-Mini	closed-source	N/A	March, 2026

The experiment is conducted as follows. Each code generation method is applied sequentially to generate code for the problems in two benchmarks using five large language models. For each problem, five code samples are generated. These samples are then compiled and tested against predefined test cases, enabling calculation of the *Comp@k*, *Pass@k*, *Pass%*, and *C-Pass%* for each method, LLM, and problem ($k=1,3$). Finally, the results are averaged across all problems to obtain the overall performance of each method on each LLM.

The results for RQ1 are shown in Table III. Rows are first grouped by five code generation approaches—VL, VL-Fix, OpenCode, SWE-Agent-Mini, and EcoreGenTL—and then further split by five distinct LLMs per approach. Columns are organized across two benchmarks, JavaBench and TLBench, with performance measured using the key metrics: *Comp@k*, *Pass@k*, *Pass%*, and *C-Pass%*.

EcoreGenTL outperforms every baseline across all benchmarks and metrics. With DeepSeek-V4-Flash on JavaBench, it scores 91.1%, well above the best baseline (74.2%) and far ahead of VL (47.4%); on TLBench, using Gemini-3.1-Flash-Lite, it reaches 98.3%. We performed paired permutation tests (10,000 resamples) on the pooled individual problems from both benchmarks, confirming that EcoreGenTL significantly outperforms every baseline ($p < 0.05$) for all five LLMs.

The choice of LLM still matters—DeepSeek and MiniMax tend to lead, while Qwen lags—but EcoreGenTL noticeably narrows the gap in *C-Pass%*. For instance, on TLBench,

DeepSeek’s *Pass%* (96.8%) far outruns Qwen’s (84.6%), yet the *C-Pass%* gap shrinks to 96.8% vs. 92.0%, implying that the method mainly helps weaker models after compilation. No single LLM dominates across all settings; the optimal pairing requires some tuning, though EcoreGenTL adapts reasonably well to different backbones.

A striking compile–run split appears in the baselines: VL-Fix with GPT-5.4-Mini gives *Comp@1* = 1.00 but *Pass@1* = 0.00, while SWE-Agent-Mini with DeepSeek returns *Pass@3* = 0.00 despite a 74.2% *Pass%*, revealing inconsistency. By contrast, EcoreGenTL delivers both high compilation and execution rates—*Pass%* and *C-Pass%* are nearly identical (e.g., 91.1% vs. 91.1%), and its *Pass@3* reaches 0.38, with the best *C-Pass%* of 97.1% on TLBench for MiniMax-M3. Thus, compilation is no longer the bottleneck; semantic correctness stays strong among compilable outputs.

Stability across LLMs also sets EcoreGenTL apart. VL and VL-Fix show wide *Pass%* spreads (4.6–47.4% and 9.1–74.8%), whereas EcoreGenTL exhibits the narrowest range (46.5–91.1%), proving it works consistently even with weaker models. Quantitatively, over VL, EcoreGenTL boosts the average *Pass%* by 47.8 points on JavaBench (from 27.4% to 75.2%) and by 15.1 points on TLBench (from 79.5% to 94.6%). Combined with the smallest gap between *Pass%* and *C-Pass%*, this confirms a dual improvement in both compilability and logical correctness—the very synergy that justifies combining MDE with LLMs.

Answer to RQ1: Based on Table III, EcoreGenTL consistently achieves the highest compilation success and test pass rates, outperforming all four baselines on both benchmarks across all five LLMs tested. It not only raises the floor for compilability but also significantly improves semantic correctness, making it a promising solution for code generation.

EcoreGenTL’s superiority stems from two complementary design choices. (1) MDE template-based code generation produces a syntactically correct and structurally complete code skeleton, avoiding common compilation errors that plague baselines. (2) LLM-driven code completion operates on this well-defined skeleton—reducing the task to filling method bodies guided by pre-generated specifications—which significantly lowers the complexity compared to generating entire classes from scratch. This synergy ensures high compilation success while substantially improving semantic correctness.

B. RQ2: Ablation Study

To answer RQ2, we compare EcoreGenTL with five ablation variants, which are described as follows.

- **EcoreGenTL w/o Req** is a variant that omits requirement decomposition and review (steps 1 and 2). In the re-

²OpenCode website: <https://opencode.ai/>.

TABLE III: Code generation performance of all tested methods

Method	LLM	JavaBench						TLBench					
		Comp@1	Comp@3	Pass@1	Pass@3	Pass%	C-Pass%	Comp@1	Comp@3	Pass@1	Pass@3	Pass%	C-Pass%
VL	MiniMax-M3	0.65	0.95	0.05	0.15	39.8%	61.3%	0.92	1.00	0.26	0.35	86.5%	92.0%
	DeepSeek-V4-Flash	0.60	0.80	0.00	0.00	47.4%	79.0%	0.98	1.00	0.34	0.45	88.1%	89.9%
	Qwen3.6-35B-A3B	0.05	0.15	0.00	0.00	4.6%	92.0%	0.74	1.00	0.16	0.25	64.3%	86.9%
	Gemini-3.1-Flash-Lite	0.30	0.40	0.00	0.00	19.2%	63.9%	0.92	1.00	0.22	0.26	82.4%	87.7%
	GPT-5.4-Mini	0.30	0.70	0.00	0.00	25.9%	86.2%	0.92	1.00	0.22	0.37	76.4%	86.8%
VL-Fix	MiniMax-M3	0.75	0.98	0.05	0.15	46.2%	61.6%	0.96	1.00	0.26	0.35	90.1%	92.0%
	DeepSeek-V4-Flash	0.80	0.90	0.00	0.00	63.7%	79.6%	1.00	1.00	0.34	0.45	90.1%	90.1%
	Qwen3.6-35B-A3B	0.15	0.38	0.00	0.00	9.1%	60.6%	0.82	1.00	0.16	0.25	73.8%	85.8%
	Gemini-3.1-Flash-Lite	0.45	0.50	0.00	0.00	21.1%	46.8%	0.92	1.00	0.22	0.26	82.4%	87.7%
	GPT-5.4-Mini	1.00	1.00	0.00	0.00	74.8%	74.8%	0.92	1.00	0.22	0.37	76.4%	86.8%
OpenCode	MiniMax-M3	0.90	1.00	0.00	0.00	68.3%	75.9%	1.00	1.00	0.30	0.30	90.2%	90.2%
	DeepSeek-V4-Flash	0.90	1.00	0.05	0.15	66.2%	75.5%	0.96	1.00	0.32	0.40	85.2%	88.8%
	Qwen3.6-35B-A3B	0.20	0.40	0.00	0.00	11.5%	57.7%	0.86	1.00	0.18	0.29	71.8%	83.4%
	Gemini-3.1-Flash-Lite	0.35	0.50	0.00	0.00	13.1%	37.5%	0.94	1.00	0.14	0.25	80.7%	85.9%
	GPT-5.4-Mini	0.85	1.00	0.00	0.00	33.3%	39.2%	1.00	1.00	0.26	0.35	85.3%	85.3%
SWE-Agent-Mini	MiniMax-M3	0.75	1.00	0.00	0.00	55.2%	73.6%	1.00	1.00	0.28	0.36	89.7%	90.3%
	DeepSeek-V4-Flash	0.90	1.00	0.00	0.00	74.2%	82.4%	0.96	1.00	0.32	0.39	86.7%	90.3%
	Qwen3.6-35B-A3B	0.35	0.65	0.00	0.00	24.1%	68.8%	0.94	0.99	0.20	0.26	78.9%	84.0%
	Gemini-3.1-Flash-Lite	0.75	0.98	0.00	0.00	32.8%	43.8%	0.90	0.96	0.20	0.20	77.6%	86.2%
	GPT-5.4-Mini	0.95	1.00	0.00	0.00	35.0%	36.9%	1.00	1.00	0.24	0.30	86.6%	86.6%
EcoreGenTL	MiniMax-M3	1.00	1.00	0.10	0.23	90.6%	90.6%	1.00	1.00	0.56	0.78	97.1%	97.1%
	DeepSeek-V4-Flash	1.00	1.00	0.15	0.38	91.1%	91.1%	1.00	1.00	0.58	0.74	96.8%	96.8%
	Qwen3.6-35B-A3B	0.65	0.98	0.00	0.00	50.5%	77.7%	0.92	1.00	0.44	0.65	84.6%	92.0%
	Gemini-3.1-Flash-Lite	0.80	0.98	0.05	0.15	70.8%	88.5%	0.96	1.00	0.56	0.65	98.3%	98.3%
	GPT-5.4-Mini	0.95	1.00	0.00	0.00	82.1%	86.4%	1.00	1.00	0.66	0.75	96.3%	96.3%

maining steps, the full requirement, rather than operation specifications, is used for code completion and fixing.

- **EcoreGenTL w/o Rev** is a variant that omits requirement review (step 2). The operation specifications derived from step 1 are used directly in later steps without correction.
- **EcoreGenTL w/o Comp** is a variant that disables code compression during code completion and fixing.
- **EcoreGenTL w/o Cont** is a variant that disables tool calling during code completion and fixing.
- **EcoreGenTL w/o Fix** is a variant that skips the code fixing (step 5).

To answer RQ2, we run ablation variants on JavaBench and TLBench using DeepSeek-V4-Flash and GPT-5.4-Mini, the two best-performing LLMs from RQ1. For each ablation variant and benchmark problem, we ask LLMs to generate five samples. We then compile and test these samples to calculate *Comp@k*, *Pass@k*, *Pass%*, and *C-Pass%* ($k=1,3$).

The results in Table IV reveal that all components contribute, but their impacts differ substantially across benchmarks depending on task complexity.

Requirement decomposition is critical for complex tasks. Removing it (w/o Req) drastically drops JavaBench’s *Comp@1* from 1.00 to 0.15 for DeepSeek and from 0.95 to 0.20 for GPT, while *Pass%* falls from 91.1% to 7.8% and from 82.1% to 5.0%, because JavaBench’s intricate logic requires explicit specifications. On TLBench, however, *Comp@1* stays at 1.00 and *Pass%* only drops to 92.2% (DeepSeek) and 94.7% (GPT), as tasks are simpler and more self-contained; the LLM can infer correct behavior from method names and the EMF-generated skeleton alone.

Specification review improves semantic correctness. Removing review (w/o Rev) drops JavaBench’s *Comp@1* from 1.00 to 0.75 (DeepSeek) and 0.95 to 0.60 (GPT); *Pass%* falls from 91.1% to 68.1% and 82.1% to 51.9%. Faulty

specifications cause syntactic and semantic errors. *C-Pass%* stays high (90.7%, 86.5%), showing that code often passes tests if it compiles, but many outputs fail to compile. On TLBench, the effect is smaller due to simpler tasks.

Code compression helps the LLM focus on relevant code. Removing compression (w/o Comp) reduces JavaBench *Comp@1* from 1.00 to 0.55 for DeepSeek and from 0.95 to 0.65 for GPT, while *Pass%* falls from 91.1% to 49.0% and from 82.1% to 48.4%. Without compression, the LLM receives full Java files, which distract the model and increase error rates. On TLBench, the effect is less pronounced because the simpler codebase contains fewer distracting elements.

Context retrieval is essential for cross-class dependencies. Without it (w/o Cont), JavaBench *Comp@1* drops to 0.00 for both LLMs—code fails to compile due to missing external type references. TLBench’s *Comp@1* stays 0.84–0.86 and *Pass%* 80%, showing its tasks are more self-contained.

Code fixing converts near-correct code into compilable code. Removing it (w/o Fix) is catastrophic: JavaBench *Comp@1* drops to 0.05 for DeepSeek and 0.00 for GPT. *C-Pass%* without fixing is actually high (96.6% for DeepSeek), indicating that when the LLM produces compilable code, it is semantically correct—but most outputs fail to compile by a small margin. On TLBench, the drop is less severe because the initial generation quality is higher and errors are fewer.

Answer to RQ2: Specification decomposition and code compression primarily enhance semantic accuracy and clarity of focus, while context retrieval tools and iterative fixing are essential for achieving compilation, particularly on complex benchmarks like JavaBench. On simpler benchmarks, each component contributes less because the tasks are less error-prone. The full pipeline outperforms every variant, demonstrating the synergy among its components.

TABLE IV: Results of ablation study

Variant	LLM	JavaBench						TLBench					
		Comp@1	Comp@3	Pass@1	Pass@3	Pass%	C-Pass%	Comp@1	Comp@3	Pass@1	Pass@3	Pass%	C-Pass%
EcoreGenTL	DeepSeek-V4-Flash	1.00	1.00	0.15	0.38	91.1%	91.1%	1.00	1.00	0.58	0.74	96.8%	96.8%
	GPT-5.4-Mini	0.95	1.00	0.00	0.00	82.1%	86.4%	1.00	1.00	0.66	0.75	96.3%	96.3%
EcoreGenTL w/o Req	DeepSeek-V4-Flash	0.15	0.38	0.00	0.00	7.8%	51.5%	1.00	1.00	0.50	0.50	92.2%	92.2%
	GPT-5.4-Mini	0.20	0.40	0.00	0.00	5.0%	24.8%	1.00	1.00	0.48	0.56	94.7%	94.7%
EcoreGenTL w/o Rev	DeepSeek-V4-Flash	0.75	0.90	0.05	0.15	68.1%	90.7%	1.00	1.00	0.46	0.64	95.7%	95.7%
	GPT-5.4-Mini	0.60	0.90	0.00	0.00	51.9%	86.5%	1.00	1.00	0.60	0.69	95.1%	95.1%
EcoreGenTL w/o Comp	DeepSeek-V4-Flash	0.55	0.75	0.00	0.00	49.0%	89.1%	1.00	1.00	0.54	0.68	94.7%	94.7%
	GPT-5.4-Mini	0.65	0.75	0.00	0.00	48.4%	74.4%	0.94	1.00	0.50	0.64	89.8%	95.5%
EcoreGenTL w/o Cont	DeepSeek-V4-Flash	0.00	0.00	0.00	0.00	0.0%	0.0%	0.84	0.89	0.46	0.55	80.1%	95.4%
	GPT-5.4-Mini	0.00	0.00	0.00	0.00	0.0%	0.0%	0.86	0.90	0.44	0.56	82.7%	96.1%
EcoreGenTL w/o Fix	DeepSeek-V4-Flash	0.05	0.15	0.00	0.00	4.8%	96.6%	0.96	1.00	0.56	0.72	92.3%	96.1%
	GPT-5.4-Mini	0.00	0.00	0.00	0.00	0.0%	0.0%	0.88	0.99	0.48	0.73	84.8%	96.3%

C. RQ3: Error Cases Analysis

To understand the typical error patterns, we examined all failed test cases of EcoreGenTL from RQ1. Two authors independently inspected each failure, localized the faulty code regions, and categorized the underlying causes. Disagreements were resolved through discussion until a consensus was reached. Notably, we aggregated faults by unique code locations rather than by failing test cases, because a single error often violates multiple test assertions simultaneously.

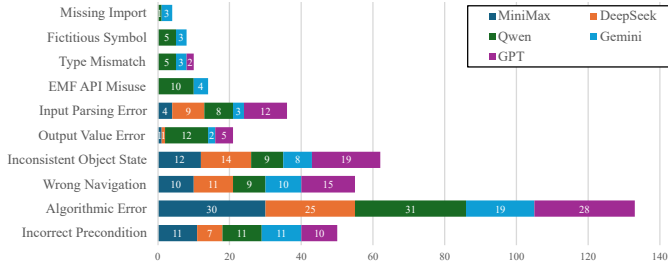


Fig. 6: Stacked distribution of fault sites across error categories per LLM

Figure 6 shows the stacked distribution of the unique fault sites across error categories per LLM. We identified 4 types of compilation errors and 6 types of runtime errors:

• Compilation Errors

- *Missing imports* refer to failures caused by the absence or incorrectness of import statements.
- *Fictitious symbols* refer to failures caused by references to non-existent types, methods, or variables.
- *Type mismatches* refer to failures arising from the passing of values between incompatible types.
- *EMF API misuses* refer to failures by the misuse of EMF-related APIs.

• Runtime Errors

- *Input parsing errors* refer to faults where the generated code fails to correctly convert raw input data (e.g., strings) into the appropriate internal data.
- *Output value errors* refer to faults where the generated code executes successfully but returns a value that deviates from the expectation.

- *Inconsistent object states* refer to faults where the code updates some fields but not the dependent ones.
- *Wrong navigation* refers to faults where the code navigates to an unintended object or field.
- *Algorithmic errors* refer to faults where the code is algorithmically incorrect.
- *Incorrect preconditions* refer to faults where the code incorrectly implements or omits precondition checks.

Based on Figure 6, algorithmic errors are the most prevalent runtime errors, accounting for 37.25% of runtime faults, followed by inconsistent object state (17.37%). Among compilation errors, EMF API misuse ranks first (38.89%), with type mismatch second (27.78%).

Comparing five LLMs, there is a sharp difference in compilation versus runtime errors. MiniMax and DeepSeek have zero compilation errors; all faults are runtime-based (68 and 67 sites). Qwen has the most compilation errors (21), followed by Gemini (13) and GPT (2). Yet, fewer compilation errors do not mean better semantics: GPT has the most runtime errors (89), and Qwen also has many (80).

Analyzing error type distributions across LLMs reveals both shared and model-specific patterns. Algorithmic errors dominate all five LLMs (30%–38% of runtime errors), highlighting a universal logic bottleneck. However, key differences appear: Qwen uniquely has missing imports and fictitious symbols, with EMF API misuse errors far higher than others (10 vs. ≤ 4). GPT shows disproportionate input parsing (12 vs. ≤ 9) and output value errors (5 vs. ≤ 2), indicating weaknesses in data transformation and return handling. For wrong navigation, GPT and Gemini lead (15 and 10), while the inconsistent object state is highest in GPT (19) and DeepSeek (14).

TABLE V: Root cause attribution of observed fault types

Fault	Requirement	LLM Code Completion	LLM Code Fix	EMF Knowledge	Incomplete Context
Missing Import	0	2	1	1	0
Fictitious Symbol	0	4	1	0	3
Type Mismatch	0	4	1	0	5
EMF API Misuse	0	0	0	14	0
Input Parsing Error	23	12	0	0	1
Output Value Error	17	4	0	0	0
Inconsistent Object State	26	13	1	18	4
Wrong Navigation	5	11	0	5	34
Algorithmic Error	33	20	4	64	12
Incorrect Precondition	41	4	0	0	5

Table V presents the root cause attribution of each fault type, where each cell indicates the number of fault sites attributed to a specific root cause. We identify the five root causes as follows: *Requirement* faults stem from decomposition; *LLM Code Completion* introduces defects during code generation from correct specs; *LLM Code Fix* accounts for errors from iterative fixing; *EMF Knowledge* faults arise from insufficient API or convention understanding; *Incomplete Context* faults come from inadequate contextual data.

Overall, *EMF Knowledge* is the top contributor across multiple categories—most notably Algorithmic Error (64), Inconsistent Object State (18), EMF API Misuse (14), and Wrong Navigation (5)—suggesting LLMs’ insufficient understanding of EMF-specific APIs is a pervasive bottleneck. *Requirement* issues are the second most dominant root cause, strongly affecting Incorrect Precondition (41), Algorithmic Error (33), Inconsistent Object State (26), and Input Parsing Error (23), indicating that vague or incorrect operation specifications often lead to semantic misunderstandings. *Incomplete Context* primarily drives Wrong Navigation (34), Algorithmic Error (12), and Type Mismatch (5), as missing cross-class or cross-method information misleads the LLM in resolving object references. *LLM Code Completion* moderately impacts most categories, especially Algorithmic Error (20), Inconsistent Object State (13), and Input Parsing Error (12), reflecting difficulty in generating complex logic. *LLM Code Fix* plays a minor role (≤ 4 each), indicating that iterative repair effectively fixes faults without introducing significant new errors.

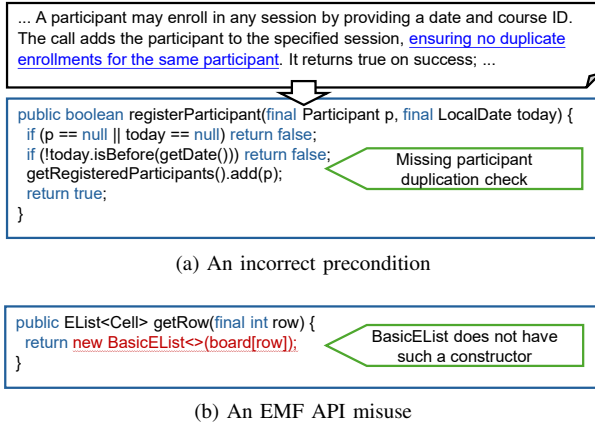


Fig. 7: Examples of fault code

Figure 7 shows two typical errors. Figure 7a shows an example of an incorrect precondition (a runtime error). The requirement mandates that “no duplicate enrollments for the same participant” be verified before registration, but the generated code omits this check. Examination of the execution log reveals that the LLM failed to identify this precondition during requirement decomposition. Despite being instructed to recognize it in the operation specification, the LLM ignored the condition and did not correct the issue in subsequent reviews. Figure 7b presents an example of an EMF API misuse. The method `getRow` aims to return a specific row’s

data from `board` as an `EList`. Since `board` is a two-dimensional `Cell` array, `board[row]` contains the desired row. The error is that `BasicEList` lacks a constructor to initialize an `EList` from an array. Due to insufficient EMF knowledge, the LLM hallucinated when generating the code.

Answer to RQ3: According to Figure 6 and Table V, the most typical errors are *Inconsistent Object States*, *Wrong Navigation*, *Algorithmic Errors*, and *Incorrect Preconditions*. Most of the errors can be attributed to incorrect requirement decomposition and insufficient EMF knowledge.

The results highlight two key findings: (1) Requirement decomposition remains a major bottleneck in code generation, with room for improvement even when structured specification and a review step are provided. (2) A core challenge in fusing EMF TBCG with LLMs is enabling LLMs to match EMF’s coding style and API knowledge. EMF knowledge is the single largest cause of residual errors, confirming that, although currently addressed in an ad-hoc manner, it is essential and warrants systematic investigation in future work.

D. Threats To Validity

Regarding *internal validity*, JavaBench was adapted from class-level Javadoc completion to model-driven generation, and TLBench was constructed by the authors. We mitigate this by preserving JavaBench’s original test cases and by evaluating on both benchmarks. Regarding *external validity*, EcoreGenTL is built on EMF/Ecore and evaluated only on Java; adapting to other frameworks or languages would require re-engineering the MDE-specific components. The evaluation is limited to 14 problems, with replication on larger benchmarks left for future work. Nevertheless, each of JavaBench’s four problems involves 13–29 classes and 32–53 methods, substantially larger than typical benchmarks [13]. EcoreGenTL’s consistent superiority across five LLMs, two benchmarks, and 14 problems offers initial evidence of generalization beyond specific model-benchmark combinations. Regarding *construct validity*, the test cases of TLBench were created by the authors, which may raise concerns about coverage adequacy. The average line coverage of these tests is 97.61%, considered acceptable.

VI. CONCLUSION

In this paper, we presented EcoreGenTL, a hybrid approach that integrates MDE template-based generation with LLM-driven completion. Our results show that templates and LLMs address distinct quality dimensions—structural correctness versus semantic completeness—and that their combination consistently outperforms pure LLM-based approaches. For the MDE community, this offers a practical path to augment existing MDE tools with LLMs without discarding proven infrastructure, and our error analysis reveals that systematic extraction of MDE coding knowledge is key to unlocking the full potential of such integration.

We plan to investigate how to automate MDE knowledge extraction from existing MDE repositories. We will also extend our approach to more languages and frameworks. Finally, we intend to apply it to larger industrial projects.

REFERENCES

- [1] A. G. Kleppe, J. Warmer, and W. Bast, *MDA Explained: The Model Driven Architecture: Practice and Promise*. USA: Addison-Wesley Longman Publishing Co., Inc., 2003.
- [2] L. Burgueño, D. Di Ruscio, H. Sahraoui, and M. Wimmer, "Automation in model-driven engineering: A look back, and ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, May 2025. [Online]. Available: <https://doi.org/10.1145/3712008>
- [3] E. Syriani, L. Luhunu, and H. Sahraoui, "Systematic mapping study of template-based code generation," *Computer Languages, Systems & Structures*, vol. 52, pp. 43–62, 2018.
- [4] Z. Hemel, L. C. L. Kats, D. M. Groenewegen, and E. Visser, "Code generation by model transformation: a case study in transformation modularity," *Software & Systems Modeling*, vol. 9, no. 3, pp. 375–402, 2010.
- [5] H. Alfraihi and K. Lano, "Trends and insights into the use of model-driven engineering: A survey," in *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 2023, pp. 286–295.
- [6] J. M. Rivero, J. Grigera, G. Rossi, E. Robles Luna, F. Montero, and M. Gaedke, "Mockup-driven development: Providing agile support for model-driven web engineering," *Information and Software Technology*, vol. 56, no. 6, pp. 670–687, 2014.
- [7] D. Akdur, V. Garousi, and O. Demirörs, "A survey on modeling and model-driven engineering practices in the embedded software industry," *Journal of Systems Architecture*, vol. 91, pp. 62–82, 2018.
- [8] E. de Araújo Silva, E. Valentin, J. R. H. Carvalho, and R. da Silva Barreto, "A survey of model driven engineering in robotics," *Journal of Computer Languages*, vol. 62, p. 101021, 2021. [Online]. Available: <https://doi.org/10.1016/j.cola.2020.101021>
- [9] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, "Low-code development and model-driven engineering: Two sides of the same coin?" *Software and Systems Modeling*, vol. 21, no. 2, pp. 437–446, 2022.
- [10] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, Dec. 2024.
- [11] S. Joel, J. Wu, and F. Fard, "A survey on llm-based code generation for low-resource and domain-specific programming languages," *ACM Trans. Softw. Eng. Methodol.*, Oct. 2025.
- [12] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A survey on large language models for code generation," *ACM Trans. Softw. Eng. Methodol.*, Jul. 2025.
- [13] J. Cao, Z. Chen, J. Wu, S.-C. Cheung, and C. Xu, "Javabench: A benchmark of object-oriented code generation for evaluating large language models," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 870–882.
- [14] Z. Zhang, C. Wang, Y. Wang, E. Shi, Y. Ma, W. Zhong, J. Chen, M. Mao, and Z. Zheng, "Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation," *Proceedings of the ACM on Software Engineering*, vol. 2, pp. 481–503, 6 2025.
- [15] Eclipse Modeling Framework. [Online]. Available: <https://eclipse.dev/emf/index.html>
- [16] DeepSeek-AI, "Deepseek-v3 technical report," 2024. [Online]. Available: <https://arxiv.org/abs/2412.19437>
- [17] J. Navarro and R. Ibarra, "Automatic test case generation using natural language processing: A systematic mapping study," *Information and Software Technology*, vol. 189, p. 107929, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S095058492500268X>
- [18] J. Zhang, X. Hu, C. Gao, X. Xia, and S. Li, "Enhancing automated unit test generation with large language models: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, Mar. 2026, just Accepted. [Online]. Available: <https://doi.org/10.1145/3802827>
- [19] Q. Zhang, C. Fang, Y. Xie, Y. Ma, W. Sun, Y. Yang, and Z. Chen, "A systematic literature review on large language models for automated program repair," *ACM Trans. Softw. Eng. Methodol.*, Mar. 2026, just Accepted. [Online]. Available: <https://doi.org/10.1145/3799693>
- [20] D. Robinson, C. Cabrera, A. D. Gordon, N. D. Lawrence, and L. Mennen, "Requirements are all you need: The final frontier for end-user software engineering," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, May 2025. [Online]. Available: <https://doi.org/10.1145/3708524>
- [21] A. C. Marcén, A. Iglesias, R. Lapeña, F. Pérez, and C. Cetina, "A systematic literature review of model-driven engineering using machine learning," *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2269–2293, 2024.
- [22] H. Naveed, C. Arora, H. Khalajzadeh *et al.*, "Model driven engineering for machine learning components: A systematic literature review," *Information and Software Technology*, vol. 169, p. 107423, 2024.
- [23] J. Di Rocco, D. Di Ruscio, C. Di Sipio, P. T. Nguyen, and R. Rubel, "On the use of large language models in model-driven engineering," *Software and Systems Modeling*, vol. 24, pp. 923–948, 6 2025.
- [24] M. Weyssow, H. Sahraoui, and E. Syriani, "Recommending metamodel concepts during modeling activities with pre-trained language models," *Softw. Syst. Model.*, vol. 21, no. 3, p. 1071–1089, Jun. 2022.
- [25] V. Kulkarni, S. Reddy, S. Barat, and J. Dutta, "Toward a symbiotic approach leveraging generative ai for model driven engineering," in *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, 2023, pp. 184–193.
- [26] M. B. Chaaben, L. Burgueño, and H. Sahraoui, "Towards using few-shot prompt learning for automating model completion," in *2023 IEEE/ACM 45th international conference on software engineering: New ideas and emerging results (ICSE-NIER)*. IEEE, 2023, pp. 7–12.
- [27] M. B. Chaaben, L. Burgueño, I. David, and H. Sahraoui, "On the utility of domain modeling assistance with large language models," *ACM Trans. Softw. Eng. Methodol.*, vol. 35, no. 4, Mar. 2026. [Online]. Available: <https://doi.org/10.1145/3744920>
- [28] K. Chen, Y. Yang, B. Chen, J. A. Hernández López, G. Mussbacher, and D. Varró, "Automated domain modeling with large language models: A comparative study," in *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, 2023, pp. 162–172.
- [29] G. B. Herwanto, "Automating data flow diagram generation from user stories using large language models," in *7th Workshop on Natural Language Processing for Requirements Engineering*, 2024.
- [30] L. Apvrille and B. Sultan, "System architects are not alone anymore: Automatic system modeling with ai," in *MODELSWARD 2024: 12th International Conference on Model-Based Software and Systems Engineering*, vol. 1. SCITEPRESS-Science and Technology Publications, 2024, pp. 27–38.
- [31] J. Cámara, J. Troya, L. Burgueño, and A. Vallecillo, "On the assessment of generative ai in modeling tasks: an experience report with chatgpt and uml," *Software and Systems Modeling*, vol. 22, no. 3, pp. 781–793, 2023.
- [32] B. Chen, K. Chen, S. Hassani, Y. Yang *et al.*, "On the use of gpt-4 for creating goal models: an exploratory study," in *2023 IEEE 31st International Requirements Engineering Conference Workshops (REW)*. IEEE, 2023, pp. 262–271.
- [33] I. Reinhartz-Berger, S. J. Ali, and D. Bork, "Leveraging llms for domain modeling: The impact of granularity and strategy on quality," in *Advanced Information Systems Engineering*. Cham: Springer Nature Switzerland, 2025, pp. 3–19.
- [34] K. Ahmed, J. Song, B. Chen, O. Wei, and B. Zheng, "Mccet: Behavioral model correctness evaluation using large language models," in *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE Press, 2025, p. 84–95.
- [35] S. L. Shrestha and C. Csallner, "Slgpt: Using transfer learning to directly generate simulink model files and find bugs in the simulink toolchain," in *Proceedings of the 25th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 260–265.
- [36] T. Sağlam, S. Hahner, L. Schmid, and E. Burger, "Automated detection of ai-obfuscated plagiarism in modeling assignments," in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training*, ser. ICSE-SEET '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 297–308.
- [37] J. A. H. López, M. Földiák, and D. Varró, "Text2vql: Teaching a model query language to open-source language models with chatgpt," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 13–24.
- [38] H. A. Siala and K. Lano, "Leveraging llms for abstracting uml and ocl representations from java and python programs," *Journal of Systems and Software*, vol. 238, 2026. [Online]. Available: <http://dx.doi.org/10.1016/j.jss.2026.112874>
- [39] A. R. Sadik, S. Brulin, M. Olhofer, A. Ceravola, and F. Joublin, "Llm as a code generator in agile model driven development," in *Model-*

Based Software and Systems Engineering, F. José Domínguez Mayo, L. Ferreira Pires, and E. Seidewitz, Eds. Cham: Springer Nature Switzerland, 2026, pp. 198–212.

- [40] N. Siavash and A. Moin, “Model-driven quantum code generation using large language models and retrieval-augmented generation,” in *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, 2025, pp. 260–266.
- [41] N. Malamas, E. Tsardoulas, K. Panayiotou, and A. L. Symeonidis, “Toward efficient vibe coding: An llm-based agent for low-code software development,” *Journal of Computer Languages*, vol. 85, 2025. [Online]. Available: <http://dx.doi.org/10.1016/j.cola.2025.101367>
- [42] Y. Yang, R. Zhang, Y. Zhu, M. Ye, M. Tang, X. Wang, and W. Wang, “Autoark: A model-based approach for arkts application code generation,” *Science of Computer Programming*, vol. 253, p. 103503, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167642326000699>
- [43] I. Al-Azzoni, S. Iqbal, T. Al Ashkar, and Z. Erum, “Integrating model-driven engineering and large language models for test scenario generation for smart contracts,” *Information*, vol. 17, no. 1, 2026. [Online]. Available: <https://www.mdpi.com/2078-2489/17/1/1>
- [44] S. Morales, R. Clariso, and J. Cabot, “Impromptu: a framework for model-driven prompt engineering,” *SOFTWARE AND SYSTEMS MODELING*, vol. 24, no. 6, pp. 1627–1645, DEC 2025.
- [45] B. Chen, K. Chene, J. A. Hernández López, G. Mussbacher, D. Varró, and A. Feizpour, “Sherpa: A model-driven framework for large language model execution,” in *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, 2025, pp. 197–208.
- [46] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, “Large language model-based agents for software engineering: A survey,” *ACM Trans. Softw. Eng. Methodol.*, Mar. 2026, just Accepted. [Online]. Available: <https://doi.org/10.1145/3796507>
- [47] R. Saini, G. Mussbacher, J. L. C. Guo, and J. Kienzle, “Machine learning-based incremental learning in interactive domain modelling,” in *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 176–186. [Online]. Available: <https://doi.org/10.1145/3550355.3552421>
- [48] F. Bozyigit, T. Bardakci, A. Khalilipour, M. Challenger, G. Ramackers, O. Babur, and M. R. V. Chaudron, “Generating domain models from natural language text using nlp: a benchmark dataset and experimental comparison of tools: Generating domain models from natural language text using nlp..” *Softw. Syst. Model.*, vol. 23, no. 6, p. 1493–1511, May 2024. [Online]. Available: <https://doi.org/10.1007/s10270-024-01176-y>
- [49] J. Yang, C. Jimenez, A. Wetteg, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 50 528–50 652.